



Stemronic AI

INTELLIGENT SOLUTIONS · INDUSTRIAL IMPACT

Lead Generation Platform

Technical Architecture & System Documentation — how the pipeline, AI scoring, data model, backend, and sales-operations layer work end to end.

Document Engineering reference (deep technical)

Product AI B2B lead generation for GCC industrial automation

Audience Technical stakeholders / engineering review

Status Live · deployed on Render with persistent storage

SECTION 01

System overview

Stemronic AI is an end-to-end platform that **discovers, enriches, scores, and works** B2B leads for industrial-automation sales across the Gulf (GCC) market. It combines an automated **AI data pipeline** that continuously sources and qualifies companies with a **sales-operations application** where managers assign leads to agents, agents call and email through the product, and every outcome — calls, replies, and RFQs — is tracked and scored.

2-step
APOLLO SOURCING

0–100
AI ICP SCORE

4 roles
RBAC + TEAMS

13.5k+
LEADS IN SYSTEM

Two halves of one system

1 · Acquisition pipeline
A Python pipeline sources companies from Apollo (and social signals), verifies contact data, and scores fit with an LLM. Runs on demand or on a schedule.

2 · Sales platform
A FastAPI + React web app where the qualified leads are assigned, worked, and measured — call console, RFQ pipeline, leaderboards, targets.

Shared spine
One SQLite/Postgres database and one REST API back both halves, so a lead flows from "discovered" to "revenue" without leaving the system.

High-level architecture

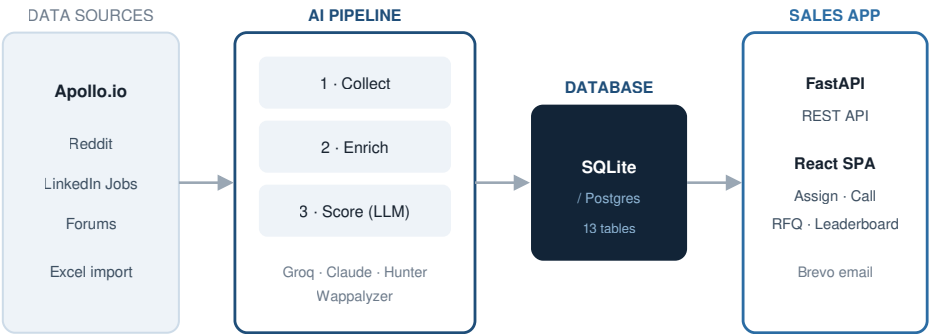


Figure 1 — Sources feed a three-stage AI pipeline; qualified leads land in one database that the sales application reads and writes.

SECTION 02

Technology stack

The system is deliberately lightweight: no heavyweight frameworks, no message queues, no external database service required to run. It is a Python backend that also serves a compiled React front-end, with a single embedded database file.

LAYER	TECHNOLOGY	NOTES
Backend / API	Python · FastAPI · Uvicorn	Async REST API; raw <code>sqlite3</code> access layer (no ORM).
Database	SQLite (default) · PostgreSQL (optional)	Driver chosen by <code>DATABASE_URL</code> . SQLite runs in WAL mode for concurrent agents.
AI — scoring & email	Groq · <code>llama-3.3-70b-versatile</code>	OpenAI-compatible endpoint; used for ICP scoring and cold-email drafting.
AI — signal extraction	Anthropic · <code>claude-sonnet-4-6</code>	Reads unstructured social/forum posts to detect buying intent.
Front-end	React 18 · Vite · Tailwind CSS	SPA with <code>framer-motion</code> , <code>recharts</code> , <code>lucide-react</code> . Built to static assets and served by FastAPI.
Lead sourcing	Apollo.io API	Two-step search + bulk enrichment; filter-rotation engine for breadth.
Contact enrichment	Hunter.io · Wappalyzer	Email finding/verification and website tech-stack detection.
Email delivery	Brevo (HTTPS API)	Transactional send over HTTPS — no SMTP ports needed (works on Render).
Deployment	Docker · Render	Multi-stage image; paid instance + 1 GB persistent disk for the database.

DESIGN PRINCIPLE

The whole product runs as **one container** and **one database file**. That keeps hosting cheap and operations simple, while the dual-driver data layer leaves a clean upgrade path to managed Postgres when scale demands it.

SECTION 03

The lead-generation pipeline

The acquisition engine is orchestrated by `main.py` and moves each lead through a status machine. A lead is a row in `raw_leads` whose `status` advances as it is processed:



Figure 2 — Lead status state machine driven by the pipeline stages.

Stage 1 — Collect `collector.py`

The primary source is Apollo, called in a cost-efficient **two-step** pattern:

- **Search (free):** `POST /api/v1/mixed_people/api_search` returns matching people IDs and previews. Filters come from `APOLLO_SEARCH`: job titles, the six Gulf locations, employee-size ranges, seniorities and email status; `per_page = 25`.
- **Bulk enrich:** matched IDs are sent to `POST /v1/people/bulk_match` in **batches of 10** (0.5 s between batches). Personal email / phone reveal is **off by default** to preserve credits.

Each result becomes a normalized lead (name, title, company, domain, LinkedIn, country, employee count, industry, email, phone). Deduplication is layered: Apollo ID → `domain + first_name` → LinkedIn URL.

SECONDARY INTENT SOURCES

Reddit (via `praw`), LinkedIn job posts (last 7 days), and engineering forums (PLCtalk, Eng-Tips) are scraped, **pre-screened** by a fast keyword check (`has_buying_signal`), then passed to Claude for structured signal extraction. Anything with `intent_level` of *none/low* is dropped before it ever reaches enrichment.

Stage 2 — Enrich `enricher.py`

- **Email finding:** Hunter.io `email-finder` returns the best email plus a verification status and confidence score.
- **Tech stack:** Wappalyzer `lookup` detects the company's website technologies (top 20 kept) — signals like SCADA/ERP vendors that indicate automation fit.

Results are written to `enriched_leads` and the lead's status becomes `enriched`. Only leads with real buying potential (`intent_level != none`) are enriched, so paid API calls are never wasted.

Stage 3 — Score `scorer.py`

Every enriched lead is scored by the LLM (detailed in Section 04) and written to `scored_leads`. Leads at or above the threshold become `queued` (sales-ready); the rest stay `scored` for reference.

SINGLE-PASS MODE

The UI drives an `apollo-only` mode that runs `collect` → `organization-enrich` → `score` in one pass, parameterised by `PIPELINE_*` environment variables. It includes a deterministic **rule-based fallback score** (title, industry, size, email, tech-stack) so a lead is always scored even if the LLM is momentarily unavailable.

SECTION 04

AI ICP scoring

The scoring model is the heart of qualification. It answers one question for every lead: **how well does this company and contact match our Ideal Customer Profile?** — and returns a defensible 0–100 score with a reason.

MODEL & CALL

Groq llama-3.3-70b-versatile via the OpenAI-compatible chat endpoint, `temperature = 0.1`, `max_tokens = 200`. The company context (`YOUR_COMPANY`) is injected into the system prompt so scoring is always relative to Stemronic's actual offering. Requests retry up to **4 times** on rate-limit (HTTP 429) with exponential backoff (2→4→8→16 s).

SCORE BANDS

SCORE	MEANING
90–100	Decision-maker (VP/Director/Manager) at a manufacturer / oil & gas / pharma company, Gulf region, 50–5,000 employees.
70–89	Engineer or senior individual contributor, right industry, Gulf region.
50–69	Partial fit.
20–49	Weak fit.
0–19	Wrong industry, student, or non-Gulf.

STRUCTURED OUTPUT

```
// strict JSON returned by the model, parsed into scored_leads
{
  "icp_score":      87,
  "intent_level":   "high",           // high | medium | low
  "offering_match": "automation",     // iot | ai | automation | multiple | none
  "reason":         "Plant manager at a 900-person GCC food manufacturer – strong automation fit."
}
```

QUALIFICATION GATE

The threshold is `MIN_ICP_SCORE = 45`. A score ≥ 45 flips the lead to `queued` (`QUEUE`); below that it is retained but not surfaced for outreach (`DROP`). `HIGH_INTENT_SCORE = 70` marks priority leads. These thresholds are configuration values, tunable without code changes.

WHY AN LLM, NOT JUST RULES

Job titles and industries are messy free text ("Head of Ops — Downstream", "Automation & Reliability Lead"). The model reads them the way a human SDR would, weighs seniority, industry, region and company size together, and explains itself — giving every score an auditable one-line rationale that agents can trust.

SECTION 05

Filter rotation & email generation

Apollo filter-rotation engine `rotation.py`

Apollo returns the same top results for the same query, so naive repeated pulls re-fetch the same companies. The rotation engine sweeps the whole market systematically instead.

- **Profiles:** one profile is generated for every **location × employee-size band** combination (e.g. "Saudi Arabia · 51–200 emp", "UAE · 201–1,000 emp"). Shared titles, industries and seniorities are copied into each.
- **Own cursor:** each profile keeps its own `page_offset`, so runs never skip or re-fetch pages.
- **Fair scheduling:** `next_profile()` always picks the **least-recently-run** profile (tie-broken by lowest offset), spreading coverage evenly across every segment of the market.
- **Isolation:** rotation state lives in its own `rotation_profiles.json`; regenerating profiles never disturbs the live pipeline settings.

AI email generation `email_gen.py`

When an agent composes outreach, the same Groq model drafts a tailored cold email (`temperature = 0.5`, `max_tokens = 400`) returning `{subject, body}` under strict rules:

- Body ≤ 3 sentences: (1) a specific reference to the company, (2) challenge → offering, (3) a low-pressure 15-minute-call CTA.
- Subject ≤ 8 words, no clickbait or exclamation marks; banned filler ("touch base", "synergy", "leverage", "I hope this email finds you well").
- Signed "The Stemronic Team"; the lead's detected tech stack is filtered to relevant keywords for context.

ALWAYS-ON FALLBACK

If no AI key is present or the model errors, a deterministic `_template()` still returns a clean subject and body — the Compose screen never shows an agent a blank draft.

SECTION 06

Data model

Thirteen tables, split cleanly between the **pipeline** (how a lead is discovered and qualified) and the **sales layer** (how it is worked and measured). The two share `raw_leads` as the spine.

PIPELINE TABLES `DATABASE.PY`

TABLE	PURPOSE & KEY COLUMNS
<code>raw_leads</code>	Master record (PK <code>id</code>). <code>source</code> , <code>first_name</code> , <code>last_name</code> , <code>title</code> , <code>company</code> , <code>domain</code> , <code>linkedin_url</code> , <code>country</code> , <code>employee_count</code> , <code>industry</code> , <code>phone</code> , <code>intent_level</code> , <code>status</code> .
<code>enriched_leads</code>	Verified contact + tech data (FK <code>lead_id</code>). <code>email</code> , <code>email_verified</code> , <code>company_size</code> , <code>industry</code> , <code>tech_stack</code> (JSON).
<code>scored_leads</code>	AI qualification (FK <code>lead_id</code>). <code>icp_score</code> , <code>intent_level</code> , <code>offering_match</code> , <code>score_reason</code> , <code>scored_at</code> .
<code>outreach_log</code>	Sent emails. <code>lead_id</code> , <code>email_subject</code> , <code>email_body</code> , <code>sent_at</code> , <code>provider_id</code> .
<code>replies</code>	Inbound replies + classification.
<code>ads_audiences</code>	High-score leads exported for ad audience matching (<code>email_hash</code> , <code>upload_status</code>).
<code>apollo_raw</code>	Raw Apollo payloads for reprocessing.

SALES-LAYER TABLES `SALES.PY`

TABLE	PURPOSE & KEY COLUMNS
<code>users</code>	Accounts & teams. <code>id</code> , <code>name</code> , <code>email</code> , <code>password</code> (salted hash), <code>role</code> , <code>manager_id</code> .
<code>lead_activities</code>	Every logged action. <code>lead_id</code> , <code>agent_id</code> , <code>type</code> , <code>outcome</code> , <code>notes</code> , <code>duration</code> , <code>interest</code> , <code>rfq_count</code> , <code>rfq_value</code> , <code>meta</code> .
<code>rfqs</code>	Opportunities. <code>lead_id</code> , <code>agent_id</code> , <code>title</code> , <code>value</code> , <code>count</code> , <code>stage</code> , <code>notes</code> , <code>updated_at</code> .
<code>tasks</code>	Follow-ups. <code>lead_id</code> , <code>agent_id</code> , <code>title</code> , <code>type</code> , <code>due_date</code> , <code>done</code> .
<code>agent_targets</code>	Per-agent daily goals (<code>daily_calls</code> , <code>daily_rfqs</code> , <code>daily_revenue</code>).
<code>call_scripts</code>	Editable talk-tracks by client category (<code>name</code> , <code>category</code> , <code>steps</code> (JSON)).
<code>activity_log</code>	Immutable audit trail of user actions.

Assignment is additive: `raw_leads` is extended with `assigned_to`, `assigned_by`, `assigned_at` via guarded migrations, so a lead's ownership lives alongside its pipeline data.

DATABASE ENGINEERING

A single access layer serves both engines. `DATABASE_URL` selects Postgres (`psycopg2`) or SQLite; placeholder and row helpers abstract the dialect difference. SQLite connections open in **WAL mode** with a 30-second busy-timeout and `synchronous = NORMAL`, allowing a whole sales team to read and write concurrently without lock errors.

SECTION 07

Backend & REST API

FastAPI exposes one JSON API and also serves the compiled React app (SPA catch-all). Endpoints fall into two groups — the pipeline/admin surface and the sales-operations surface.

CORE & PIPELINE API_SERVER.PY

AREA	ENDPOINTS
Auth & session	POST /auth/login · logout · GET /auth/me · POST /auth/password
Leads	GET /leads · GET /leads/{id}/detail · POST /leads · DELETE /leads/{id} · POST /leads/{id}/score
Import	GET /leads/template · POST /leads/import (Excel/CSV)
Pipeline control	GET/POST /pipeline/settings · POST /pipeline/{step} · GET /pipeline/status · GET /pipeline/rotation · POST /pipeline/rotation/regenerate
Email	GET /email/generate · GET /email/status · POST /email/send (Brevo)
Analytics	GET /analytics/overview · segments · economics
Users / RBAC	GET/POST /users · PUT /users/{id}/role · DELETE /users/{id}

SALES OPERATIONS SALES_API.PY

AREA	ENDPOINTS
Assignment	POST /leads/{id}/assign · POST /leads/assign (bulk)
Activities	POST /leads/{id}/activity · GET /leads/{id}/activities · POST /leads/{id}/call
RFQ pipeline	GET /rfqs · POST /rfqs · PUT /rfqs/{id}/stage
Tasks & cadences	GET /tasks/mine · POST /tasks · POST /tasks/{id}/complete · GET /cadences · POST /leads/{id}/enroll
Call scripts	GET/POST /call/scripts · PUT/DELETE /call/scripts/{id}
Performance	GET /performance/leaderboard · agent/{id} · daily · GET/POST /performance/weights · GET/POST /targets

SECTION 08

Sales-operations layer

This layer turns a list of qualified leads into a managed sales floor — who works what, what happened on each touch, and how every agent is performing.

ROLES & TEAMS (RBAC)

Four roles gate every action through a permission map: `admin` `manager` `agent` `viewer` . Managers own agents via `manager_id` (teams). Permissions cover `view` · `edit` · `delete` · `send` · `manage` · `assign` · `log`.

WORKING A LEAD

Managers assign leads (individually or in bulk, with re-assignment guards). Agents open the **guided Call Console** — a timer, category-specific talk-track, live notes, a 1–10 interest rating, and RFQ capture. Logging a call writes an activity, an audit entry, an optional follow-up task, and — if an RFQ was secured — creates an opportunity in the pipeline automatically.

RFQ PIPELINE & WEIGHTED FORECAST

RFQs move through four stages, each carrying a probability weight used for the forecast:

STAGE	NEW	QUOTED	WON	LOST
Forecast weight	0.30	0.60	1.00	0.00

Weighted forecast = $\sum$ (RFQ value × stage weight), giving managers a realistic revenue projection rather than a raw sum.

EFFORT + OUTCOME LEADERBOARD

Agents are scored on a transparent weighted-points model (configurable in-app) — RFQs are the outcome that matters most:

RFQ SECURED	PER \$1K RFQ VALUE	REPLY	CALL	EMAIL
40	+1	15	5	3

Managers set per-agent daily **targets** (calls, RFQs, revenue) that drive target-vs-actual charts, and can enrol leads into multi-touch **cadences** (Standard 9-day, Fast 4-day, Nurture) that auto-schedule the next task.

SECTION 09

Security, deployment & operations

Security & authentication

- **Passwords** are stored as salted SHA-256 hashes (`salt$hash`), never in plaintext.
- **Sessions** are bearer tokens issued on login and checked on every request; role permissions are enforced server-side, not just hidden in the UI.
- **Secrets** (Apollo, Groq, Brevo keys) live only in `.env` / host environment variables — git-ignored and docker-ignored, never committed.
- **Least privilege** — viewers read, agents work only their own leads, managers manage their team, admins manage the platform.

Deployment & persistence

The app ships as a **multi-stage Docker image**: stage 1 builds the React front-end with Node; stage 2 runs the FastAPI server and serves those static assets. It is deployed on **Render** (paid instance) with a **1 GB persistent disk** mounted at `/var/data`, so the SQLite database survives every restart and redeploy.

FIRST-BOOT SEEDING

On first start against an empty disk, the app copies a baked-in baseline database onto persistent storage, then writes there forever after. Seeded team accounts are created idempotently, so the real roster always exists — even after a reset.

- **Concurrency**: SQLite WAL + 30 s busy-timeout supports simultaneous agents.
- **Email transport**: Brevo over HTTPS (no SMTP ports), which is required on Render's network.
- **Backups**: Render takes automatic daily disk snapshots retained 7+ days.
- **Scale path**: set `DATABASE_URL` to migrate to managed Postgres with no application rewrite.

How a lead becomes revenue — end to end

Apollo search → bulk enrich → Hunter email + Wappalyzer tech → LLM ICP score → **queued** → manager assigns to an agent → agent calls via the Console → interest rated, RFQ captured → RFQ advances new → quoted → **won** → agent's leaderboard points and the team's weighted forecast update in real time. One system, one record, discovery to deal.